ECE 3218 Lab #2 - Multiple Precision Arithmetic

Introduction

The native data size of the AVR ATmega328P is 8 bits. That greatly restricts the precision of arithmetic that can be done with single machine instructions. Many uses of the controller require arithmetic on numbers much larger than 0…255 unsigned or -128…127 signed. That requires multi-precision arithmetic. In this lab, you will write and test programs to perform 32-bit addition and subtraction and 32x32-bit to 64-bit multiplication. All three will be for both unsigned and signed numbers.

The routines will be written as subroutines with arguments passed on the stack. This is a more general method that does not have the limitations of passing data in registers or fixed memory locations. It works as follows: before the subroutine call, push all the data the subroutine will need onto the stack. If you want to pass pointers to data, remember that the stack can hold addresses as well as data. Inside the subroutine, pull the data off of the stack and process it. When finished, push the "answers" onto the stack and return. The calling program should retrieve the "answers" from the stack and continue. The challenge is keeping up with the state of the stack and the order of the pushes and pulls. Don't forget that `call` and `ret` have their own effect on the stack.

As you know the data segment cannot be preloaded with data. You have to enter the data by hand. This could be tedious however, so we will use a better method to do it: we will prepare data in the code segment and then use a subroutine to copy the bytes into the data segment. Study the instruction `lpm` (Load Program Memory). Important note: the Z register must point into the code segment, but the code segment addresses are *word* (2-byte) addresses, as opposed to *byte* addresses. This means that a word address of 0x04 would correspond to a byte address of 0x08. To convert a word address to a byte address, simply multiply the word address by 2. The code below shows how a byte from the code segment can be loaded into `r0`:

```
.cseg
   …
   ldi   zh,high(const*2)     ; Note the *2!
   ldi   zl,low(const*2)
   lpm                  ; r0 gets 0x13
   …
const:  ; also in cseg
   .db   0x13
```

The next page is taken from "Atmel ATmega328P - doc0856 - Instruction Set (Detailed)."

LPM – Load Program Memory

Description:

Loads one byte pointed to by the Z-register into the destination register Rd. This instruction features a 100% space effective constant initialization or constant data fetch. The Program memory is organized in 16-bit words while the Z-pointer is a byte address. Thus, the least significant bit of the Z-pointer selects either low byte (Z_{LSB} = 0) or high byte (Z_{LSB} = 1). This instruction can address the first 64K bytes (32K words) of Program memory. The Z-pointer Register can either be left unchanged by the operation, or it can be incremented. The incrementation does not apply to the RAMPZ Register.

Devices with Self-Programming capability can use the LPM instruction to read the Fuse and Lock bit values. Refer to the device documentation for a detailed description.

The LPM instruction is not available in all devices. Refer to the device specific instruction set summary.

The result of these combinations is undefined:

```
LPM r30, Z+
LPM r31, Z+
```

	Operation:		Comment:
(i)	R0 ← (Z)		Z: Unchanged, R0 implied destination register
(ii)	Rd ← (Z)		Z: Unchanged
(iii)	Rd ← (Z)	Z ← Z + 1	Z: Post incremented

	Syntax:	Operands:	Program Counter:
(i)	LPM	None, R0 implied	PC ← PC + 1
(ii)	LPM Rd, Z	0 ≤ d ≤ 29	PC ← PC + 1
(iii)	LPM Rd, Z+	0 ≤ d ≤ 29	PC ← PC + 1

16-bit Opcode:

(i)	1001	0101	1100	1000
(ii)	1001	000d	dddd	0100
(iii)	1001	000d	dddd	0101

Status Register (SREG) and Boolean Formula:

I	T	H	S	V	N	Z	C
–	–	–	–	–	–	–	–

Example:

```
    ldi   ZH, high(Table_1<<1); Initialize Z-pointer
    ldi   ZL, low(Table_1<<1)
    lpm   r16, Z              ; Load constant from Program
                              ; Memory pointed to by Z (r31:r30)
...
Table_1:
.dw 0x5876                    ; 0x76 is addresses when Z_lsb = 0
                              ; 0x58 is addresses when Z_lsb = 1
...
```

Before the Lab:

Write programs to do the following:

1. In your **notebook:** draw a flowchart for the task in #2 below.
2. Write a subroutine that copies a number of bytes from the code segment to the data segment. Assume `Z` points to the code segment, `X` points to the data segment, and `r16` contains the number of bytes. You will use this subroutine later to initialize the data segment with test data. Here is a code snippet that sets up the registers and then calls the subroutine (labelled 'copybytes'.)

```
; get data from program memory (two 4-byte numbers)

   ldi        zh,high(pmv1*2)      ; list in program memory
   ldi        zl,low(pmv1*2)
   ldi        xh,high(value1)      ; list in sram (data memory)
   ldi        xl,low(value1)
   ldi        r16,0x08             ; number of bytes in list
   rcall      copybytes
```

   Hard code two positive 32-bit (4-byte) numbers into program memory at the end of your code. Use your subroutine to transfer them to the data segment where you have reserved two labeled 4-byte spaces. **Important**: Store 32-bit numbers in memory with the MSB in the low address and the LSB in the low address+3. Test your code in the simulator to ensure it is working properly.

2. Write a subroutine to add two 32-bit numbers. Don't just start typing at the computer! Work out the algorithm by hand first. Work a few specific examples first, and then work out the general solution. **All this should be included in your lab notebook.** The numbers to be added should be pushed onto the stack by the calling routine. The result should be placed on the stack by the subroutine. Here is some example code that pushes data on the stack in preparation for calling the subroutine:

```
; push the numbers
   ldi        xh,high(value1)      ; [x] <- addr(1st value MSB)
   ldi        xl,low(value1) ;
   ldi        r16,8                ; countdown for two 4-byte
                                   ; numbers.
pushloop:
   ld         r0,x+                ; get values 1 and 2 from
                                   ; data memory
   push       r0              ; and push them on the stack.
   dec        r16             ; MSB is first in.
   brne       pushloop        ; continue until 8 bytes are done.
   rcall      add32bit
```

Here is starter code for the subroutine that pulls the data off the stack so that the two numbers can be added. This example temporarily places the data from the stack into local data memory, where it can be accessed to add together:

```
; subroutine
; arguments are pass by value
; IN: stack (LSB to MSB): 1st number, 2nd number
; OUT: stack (LSB to MSB): sum
; status register is set according to the result
add32bit:
   ; save return address
   pop   r19              ; LSB of return address
   pop   r18              ; MSB of return address
   ; get the numbers off the stack and into local memory
   ldi   xh,high(a32v1+8)      ; [x] <- addr(1st value LSB+1)
   ldi   xl,low(a32v1+8) ;
   ldi   r16,8            ; byte count-down
poploop:                       ;
   pop   r0               ; pop byte off stack
   st    -x,r0            ; store it in data memory
   dec   r16              ; and continue until 8 bytes are done
   brne  poploop          ;
```

After the starter code, add your own code that adds the two numbers together. If you are using the example code, you will need to load the appropriate bytes from memory, add them together, and store the result in memory. Make sure to keep track of the carry bit!

Immediately after the addition is performed the status register is preserved. The sum is to be returned on the stack. The example code here loads the resulting sum from the memory location ‘a32sum’ and pushes it onto the stack before returning:

```
   in    r4,sreg          ; preserve sreg
   ldi   xh,high(a32sum)  ; [z] <- addr(sum MSB)
   ldi   xl,low(a32sum)   ;
   ldi   r16,4            ; number of bytes to push
pushloop2:                ;
   ld    r0,x+            ; get sum byte from data memory
   push  r0               ; and push it on the stack
   dec   r16              ; MSB is first in
   brne  pushloop2        ; continue until 4 bytes are done
   out   sreg,r4          ; restore the status register
   push  r18              ; MSB of return address
   push  r19              ; LSB of return address
   ret
```

Because the code above preserved the status register after the addition then restored it before returning from the subroutine the status register indicates the result of the final addition – the `S`, `V`, `N`, `Z`, and `C` bits. None of the data transfer instructions, branch, jump, call, or return instructions change the status register. Any math, logical, bit-set, test, or compare instructions do affect the status register. If your routine performs any of those after the final byte-wise addition, you will need to save the status register immediately after the final add and restore it before the return from subroutine. The status register can be saved in or restored from register `Rn` by

```
in    Rn,SREG    ;save the status register in register n
```

and

```
out   SREG,Rn    ;load the status reg from register n
```

3. Write code to retrieve two 4-byte numbers from data memory, and push them on the stack. Then call your 32-bit add routine to sum the two numbers. Pull the result off the stack and store it in another 4-byte space in data memory that you reserved for the sum. Make sure that the MSB of the sum is in the low address. (Extra credit: Write code to check the status register to determine if there was a carry out and set another data location accordingly as a flag.) Your main program should push the data, call the subroutine, and then retrieve the answer from the stack. Meanwhile, the subroutine should retrieve the input numbers from the stack and store the answer back before it returns. Note carefully the description of the 'call' and `ret` instructions and their effect on the stack before you use the stack in a subroutine. There are some adjustments you must make. To make testing easier, have your main program read the input numbers from memory and store the results back in memory after the sum.
4. Write a subroutine to subtract two 32-bit numbers. You can reuse your add routine to do this. In your subtract routine, take the two's complement of the subtrahend then call the 32-bit add routine
5. Test the subtract routine as you did with the addition.
6. Write a subroutine that will multiply two 32-bit binary numbers and produce a 64-bit result. For this program, rather than pushing on the stack the numbers to be operated on, push pointers to the two numbers to be multiplied and their product. Begin by writing a subroutine that multiplies two single byte numbers and returns a two-byte output. Once you have written this subroutine, feel free to use the `mul` instruction when writing your subroutine for the full 32-bit multiplication. Your final multiplication subroutine must be able to handle both signed and unsigned multiplication.
7. Test the multiply routine.

A word on testing –

You, the engineer, are responsible for testing your own program to ensure that it meets all of the specifications. A single, casual test is usually not sufficient. Create data sets for testing that will exercise the full range of the program's capabilities. If a specification has upper and lower limits, test your program at those limits. For this lab, determine what the limits are for each program. Create a data set to test the programs to these limits. Predict the results. Have the data with answers ready before the lab so you can be productive when you are ready to test.

Materials for lab - Bring the following items with you to lab:

1. Your laptop with the above programs typed, and compiled, already tested.
2. Written program limitations and test data sets with answers.
3. Algorithm for multiplication if code is not completed

In the Lab

Assemble your programs on the PCs in the lab using the Microchip Studio. Be sure that you have selected the listing option for the assembler. You will demonstrate your working programs for the lab TA.

Load and run your first program for this lab. Display the memory that holds the numbers. Also display the local storage areas in your subroutine, and the content of the stack. By the way, where is the stack? Who put it there? Is this where it is on the Arduino board? Can we move it? How? Single step your program through the first number raised to a power. What happens to the stack when the call instruction is executed? What happens on ret? Debug your program (if necessary) and run your test cases. Verify the results.

Record (draw a diagram) of the stack at the following points for one subroutine call:

1. **before the parameter push**
2. **before the subroutine call**
3. **after the subroutine call**
4. **after the subroutine pulls the parameters**
5. **after the subroutine pushes the result**
6. **before the `ret`**
7. **after the `ret`**
8. **after the main retrieves the result**

Record the memory contents and addresses, and the value of the stack pointer at each point. Run your test data and verify the results.

After the Lab

Produce clean and well commented listing of the code you used in the lab. Provide a table of the test data sets and results and describe the rationale behind the testing (i.e. why did you choose these numbers?). Answer briefly the following questions:

What are the benefits of a simulation vs. a 'real' AVR? What are the disadvantages?

In your programming class you learned about pass-by-value vs. pass-by-reference for subroutine arguments. Which method did you use in the first two programs? What about the third? Comment on the relative complexity of both methods.

How is the stack initialized on the Arduino board? How is it initialized in the simulator? Where is the 'system,' stack on the simulator?